

1. 본인확인-API 암호화용 키 정보 파일에서 정보 가져오기

1 본인확인 암호화용 키 정보 (mok_keyInfo.dat) 파일에서 정보 가져오기



중요

키파일은 서버 내 안전한 장소에 보관

서비스 등록 후 발급된 mok_keyInfo.dat 를 안전한 서버 내 디렉토리에 보관

Step 1. 암호화된 키 파일 준비

- **Step 1-1.** 서비스 등록 후 발급받은 본인확인 암호화 키 정보(예 : mok_keyInfo.dat)파일을 준비합니다.
- **Step 1-2.** 암호화 키 정보 파일을 byte[] 배열로 읽어서 준비합니다.

❗ 키 파일은 본인확인 인증결과 데이터를 복호화하기 위한 암호화된 정보를 포함합니다.

Step 2. AES 복호화 키 및 IV 생성

- **Step 2-1.** 암/복호화를 위한 해시함수에 SHA-256 해시 인스턴스를 생성합니다.
- **Step 2-2.** mobileOK_password(키 파일 비밀번호)를 해싱합니다.
 - Hash1(AES_KeyBytes) : mobileOK_password(키 파일 비밀번호)의 byte를 가지고 해싱합니다.
 - 해싱 결과의 앞 16바이트를 AES_KeyBytes (32byte의 앞16byte)로 사용합니다.
- **Step 2-3.** 해싱을 한 번 더 실행하여 IV(초기화 벡터) 및 AES 키의 나머지 부분을 생성합니다.
 - Hash2 : 해싱 결과(Hash1)을 한번 더 해싱합니다.
 - AES_KeyBytes (Hash1) : Hash1의 앞 16byte + Hash2의 뒤 16byte
 - AES_IvBytes (초기화 벡터 16byte) : Hash2의 앞 16byte

❗ 이 과정은 mok_keyInfo.dat 파일을 복호화하기 위해 AES 256bit 키와 16byte IV를 생성하는 단계입니다. 키 파생에는 SHA-256 해시를 2회 적용하여 키와 IV를 분리 생성합니다.

- AES_KeyBytes: 32바이트 AES 키 (Hash1 앞 16바이트 + Hash2 뒤 16바이트).

- AES_IvBytes: 16바이트 초기화 벡터 (Hash2 앞 16바이트).

Step 3. 데이터 복호화

- **Step 3-1.** AES/CBC/PKCS5Padding 모드로 위에서 생성한 AES 키와 IV를 이용하여 데이터를 복호화합니다.

Step 4. 결과 확인

- **Step 4-1.** 복호화된 결과를 JSON, UTF-8 문자열로 변환



중요

JSON 형태로 복호화가 되는지 검증 필수

```
{
  "ServiceId" : "7a2d276f-2bd0-4b60-852b-b91da40dd9d3",
  "ClientPrivateKey" : "MIIEvAIBADANBgkqhkiG9w0BAQE ... TcWg==",
  "ServerPublicKey" : "MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ ... 8DAQAB"
}
```



Info

ServiceId : 이용기관 키파일의 고유 ID (인증 요청 시 서비스 식별자로 활용됨)

ClientPrivateKey : 개인정보 복호화 시 사용됨 (→ 5. 본인확인-표준창 개인정보 복호화 참고)

ServerPublicKey : 거래요청 시 클라이언트 정보 암호화에 사용됨 (→ 2. 본인확인-표준창 거래요청정보 생성 참고)

샘플은 JAVA로 제공됩니다.

```
public class MokKeyInfoDecryption {

    public static String decryptMokKeyInfo(String filePath, String mobileOKPassword) throws Exception {
        // Step 1: 키 파일 준비
        // Step 1-1 & 1-2: mok_keyInfo.dat 파일을 byte[]로 읽기
    }
}
```

```

byte[] encryptedData = Files.readAllBytes(Paths.get(filePath));

// Step 2: AES 복호화 키 및 IV 생성
// Step 2-1: SHA-256 해시 인스턴스 생성 |
MessageDigest sha256 = MessageDigest.getInstance("SHA-256");

// Step 2-2: mobileOK_password 해싱 (Hash1)
byte[] passwordBytes = mobileOKPassword.getBytes(StandardCharsets.UTF_8);
byte[] hash1 = sha256.digest(passwordBytes); // 32바이트 해시 결과
byte[] aesKeyBytes = new byte[32]; // AES 키를 위한 32바이트 배열 준비
System.arraycopy(hash1, 0, aesKeyBytes, 0, 16); // Hash1의 앞 16바이트를 AES 키 앞부분으로 사용

// Step 2-3: 두 번째 해싱 (Hash2)으로 IV와 AES 키 나머지 부분 생성
byte[] hash2 = sha256.digest(hash1); // Hash1을 다시 해싱
System.arraycopy(hash2, 16, aesKeyBytes, 16, 16); // Hash2의 뒤 16바이트를 AES 키 뒷부분으로 사용
byte[] aesIvBytes = Arrays.copyOfRange(hash2, 0, 16); // Hash2의 앞 16바이트를 IV로 사용

// Step 3: 데이터 복호화
// Step 3-1: AES/CBC/PKCS5Padding 모드로 복호화
Cipher aesCipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
SecretKeySpec secretKeySpec = new SecretKeySpec(aesKeyBytes, "AES");
IvParameterSpec ivSpec = new IvParameterSpec(aesIvBytes);
aesCipher.init(Cipher.DECRYPT_MODE, secretKeySpec, ivSpec);

byte[] decryptedData = aesCipher.doFinal(encryptedData);

// Step 4: 결과 확인
// Step 4-1: 복호화된 결과를 UTF-8 JSON 문자열로 변환
String jsonResult = new String(decryptedData, StandardCharsets.UTF_8);

// JSON 검증 (간단히 출력으로 확인, 실제로는 JSON 파싱 후 검증 필요)
System.out.println("Decrypted JSON: " + jsonResult);
return jsonResult;
}

// 테스트용 메인 메서드
public static void main(String[] args) throws Exception {
    String filePath = "path/to/mok_keyInfo.dat"; // 실제 파일 경로로 변경
    String mobileOKPassword = "your_mobileOK_password"; // 실제 비밀번호로 변경

    String decryptedResult = decryptMokKeyInfo(filePath, mobileOKPassword);

```

1. 본인확인-API 암호화용 키 정보 파일에서 정보 가져오기

```
// JSON 형태로 복호화되었는지 검증 (예: ServiceId, ClientPrivateKey, ServerPublicKey 포함 여부 확인)
ObjectMapper mapper = new ObjectMapper();
Map<String, String> result = mapper.readValue(decryptedResult, Map.class);
if (result.containsKey("ServiceId") && result.containsKey("ClientPrivateKey") && result.containsKey("ServerPublicKey")) {
    System.out.println("JSON 복호화 성공");
} else {
    System.out.println("JSON 복호화 실패: 올바른 JSON 형식이 아님");
}
}
```

2. 본인확인-API 거래요청정보(토큰) 생성

2 본인확인 거래요청정보(토큰) 생성

Summary

clientTxId 는 본인확인 요청과 결과를 매핑하고 재사용을 방지하기 위한 **고유 식별자**입니다.
이 값은 요청과 응답의 일치성을 검증하고, 인증 결과 재사용을 방지하는 데 사용됩니다.

Step 1. 이용기관 거래 ID (clientTxId) 생성

- **Step 1-1.** 중복되지 않는 이용기관의 거래ID (clientTxId)를 생성합니다.

이용기관 거래 ID 생성 규칙

- 거래ID는 최소 20자 이상 40자 이내로 생성
- 본인확인 이용기관 거래ID 는 유일한 값이어야 하며 기 사용한 거래ID가 있는 경우 오류 발생
- 이용기관이 고유식별 ID로 유일성을 보장할 경우 고객이 이용하는 ID사용 가능

예시) [이용기관 식별자] + [UUID]

```
"ORG001-550e8400-e29b-41d4-a716-4466554"
```

인증 결과 검증을 위한 거래ID 세션 저장

- 동일한 세션 내 요청과 결과가 동일한지 확인 및 인증결과 재사용 방지처리, 응답결과 처리 시 필수 구현
- 세션 내 거래ID를 저장하여 검증하는 방법은 권고 사항이며, 이용기관의 저장매체(DB 등)에 저장하여 검증 가능

clientTxId 세션 저장

- `clientTxId` 는 인증 요청과 결과 프로세스의 일치성을 검증하기 위해 세션 또는 이용기관의 저장매체(DB 등)에 저장에 저장해야 합니다.
- 예) 세션 저장

```
HttpSession session = request.getSession();
session.setAttribute("clientTxId", clientTxId);
session.setMaxInactiveInterval(10 * 60); // 10분 만료
```

- 검증 완료 후 `clientTxId` 는 즉시 사용 완료 처리(삭제)하여 재사용을 방지합니다.

Step 2. 본인확인 토큰요청 데이터 생성

- **Step 2-1.** 암호화 전 데이터 생성 (JSONData 생성)합니다.
- **Step 2-2.** JSON 데이터 직렬화

📄 JSONData 생성 및 직렬화 규칙

이용기관 거래 ID(clientTxId), 버전, 현재 요청 시간을 포함하는 JSON 객체를 생성하고, 이를 문자열로 직렬화합니다.

- version : "V2"
- clientTxId : 이용기관 거래 ID (예 : ORG001-550e8400-e29b-41d4-a716-44665544)
- requestTime : 현재 요청 시간 (형식 : yyyyMMddHHmmss, 예: 20250318113000)
- JSON 객체를 문자열로 직렬화하여 암호화에 사용할 텍스트 데이터를 준비합니다. (예: JSON 직렬화 라이브러리 사용, UTF-8 인코딩)

예시)

```
{
  "version" : "V2",
  "clientTxId" : "ORG001-550e8400-e29b-41d4-a716-44665544",
  "requestTime" : "20250318113000"
}
```

- **Step 2-3.** 암호화 진행(RSA 암호화) 합니다.

1. RSA 서버 공개키 준비

- 공개키를 Base64 디코딩 하여 바이트 배열(byte[]) 형태로 읽어서 준비

2. RSA 암호화 방식 설정

- RSA/ECB/OAEPWithSHA-256AndMGF1Padding 모드(OAEP: SHA-256, MGF1: SHA-256)로 위에서 생성한 데이터(직렬화된 데이터)를 암호화

3. Base64 인코딩 처리

- 암호화된 데이터를 Base64로 인코딩하여 전송 가능한 문자열 형태로 변환

 Warning

RSA-OAEP 평문 길이 제한(2048-bit 기준 약 190B 전후)

 Info

- serverPublicKeyBase64 는 **1** 본인확인 암복호화용 키 정보 가이드에서 mok_keyInfo.dat 복호화로 얻은 **ServerPublicKey(X.509)** 값을 사용합니다. (요청 암호화용)
- 공개키는 X.509 포맷으로 관리됨

Step 3. 결과 확인

- **Step 3-1.** Base64 인코딩된 본인확인 요청 토큰(encryptReqClientInfo)

```
"WBIF9sY4qATKJD+CrXmx3Ml7 ... cf6z+a5pX+pvebiFmt6ChieUnpZp1pLA=="
```

샘플은 JAVA로 제공됩니다.

```
public class MOKTokenRequestEncryption {

    public static String generateEncryptReqClientInfo(String serverPublicKeyBase64) throws Exception {

        // Step 1: 이용기관 거래 ID (clientTxId) 생성
        // Step 1-1: 고유한 거래 ID 생성 (예: ORG001 + UUID)
        String identifier = "ORG001";
        String uuid = UUID.randomUUID().toString().replaceAll("-", "").substring(0, 32); //UUID에서 32자리 추출
        String clientTxId = identifier + "-" + uuid; // 최소 20자, 최대 40자 내로 생성
        if (clientTxId.length() < 20 || clientTxId.length() > 40) {
            throw new IllegalStateException("clientTxId length must be between 20 and 40 characters");
        }
    }
}
```

```

}

// Step 2: 본인확인 토큰요청 데이터 생성
// Step 2-1: JSON 데이터 생성
ObjectMapper mapper = new ObjectMapper();
Map<String, String> data = new HashMap<>();
data.put("version", "V2");
data.put("clientTxId", clientTxId);
data.put("requestTime", new SimpleDateFormat("yyyyMMddHHmmss").format(new Date()));

// Step 2-2: JSON 데이터 직렬화
String jsonData;
try {
    jsonData = mapper.writeValueAsString(data);
} catch (Exception e) {
    throw new RuntimeException("JSON 직렬화 실패", e);
}

// Step 2-3: RSA 암호화
// RSA 서버 공개키 준비
byte[] publicKeyBytes = Base64.getDecoder().decode(serverPublicKeyBase64);
X509EncodedKeySpec keySpec = new X509EncodedKeySpec(publicKeyBytes);
KeyFactory keyFactory = KeyFactory.getInstance("RSA");
PublicKey publicKey = keyFactory.generatePublic(keySpec);

// RSA-OAEP(SHA-256, MGF1-SHA-256)로 암호화
OAEPParameterSpec oaepParameterSpec = new OAEPParameterSpec("SHA-256", "MGF1", MGF1ParameterSpec.SHA256, PSource.PSpecified.DEFAULT);
Cipher rsaCipher = Cipher.getInstance("RSA/ECB/OAEPWithSha-256AndMGF1Padding");
rsaCipher.init(Cipher.ENCRYPT_MODE, publicKey, oaepParameterSpec);
byte[] encryptedData = rsaCipher.doFinal(jsonData.getBytes(StandardCharsets.UTF_8));

// Base64 인코딩 처리
String encryptReqClientInfo = Base64.getEncoder().encodeToString(encryptedData);

// Step 3: 결과 확인
System.out.println("Encrypted Request Client Info: " + encryptReqClientInfo);
return encryptReqClientInfo;
}

// 테스트용 메인 메서드
public static void main(String[] args) throws Exception {

```


2. 본인확인-API 거래요청정보(토큰) 생성

```
// 1 본인확인 암호화용 키 정보 파일에서 정보 가져오기 가이드에서 ServerPublicKey 획득
String serverPublicKeyBase64 = "serverPublicKeyBase64";
String encryptedResult = generateEncryptReqClientInfo(serverPublicKeyBase64);
System.out.println("Final Encrypted Result: " + encryptedResult);
}
}
```

3. 본인확인-API 토큰 발급 요청

토큰 발급 요청

생성된 본인확인 거래요청정보(토큰) encryptReqClientInfo 를 이용하여 본인확인 서버로 토큰 발급 요청합니다.

- 본인확인서비스 거래토큰은 최초 생성 후 본인확인-API 인증요청이 실행되면 재사용 할 수 없음
- 거래토큰 유효시간은 총 10분으로 인증 요청 후 7분내에 인증처리(이동통신사 기준) 되어야 하며 인증번호 2번 재시도 가능
- 10분이 넘어가면 새로운 거래토큰을 생성하여 처음부터 처리

본인확인-API 거래토큰 연동 URL

메서드	URL	환경
POST	https://scert-dir.mobile-ok.com/agent/v2/token/get	개발
POST	https://cert-dir.mobile-ok.com/agent/v2/token/get	운영

1 요청

헤더

- HTTPS 암호화 통신 : TLS 1.2 이상

이름	설명	필수
Content-Type	Content-Type : application/json; charset=UTF-8 요청 데이터 타입	O

본문

이름	타입	설명	필수
serviceId	String	본인확인 이용기관 서비스 ID (본인확인 암호화용 키 정보 내 포함됨)	O
encryptReqClientInfo	String	암호화된 본인확인 거래 요청 정보	O
siteUrl	String	본인확인서비스 요청 이용기관 등록 도메인	O

2 응답

본문

이름	타입	설명	필수
encryptMOKToken	String	암호화된 본인확인-API 거래 토큰	O
publicKey	String	사용자 정보를 암호화하기 위한 휴대폰본인확인서버 암호화용 공개키 * 암호알고리즘은 RSA-OAEP를 사용	O
resultCode	String	응답 결과 코드 -2000일 경우 성공 -2000이 아닐 경우 에러 처리	O
resultMsg	String	응답 결과 코드가 2000이 아닐 경우 에러 메시지	O

예제

요청

```
curl -X POST https://scert.mobile-ok.com/agent/v2/token/get \
-H "Content-Type: application/json; charset=UTF-8" \
-d '{
    "serviceId" : ${serviceId},
```

```
"encryptReqClientInfo" : ${encryptReqClientInfo},  
"siteUrl" : ${siteUrl}  
'
```

응답 : 성공

```
{  
  "encryptMOKToken" : "[RBj8YiVJ5e8lOChfhE76+AJj ... pikDZMbeegv9r79uVUcMdA==",  
  "publicKey" : "MIIBIjANBgkqhkiG ... N+Ey0zPfswIDAQAB",  
  "resultCode" : "2000",  
  "resultMsg" : "success"  
}
```

4. 본인확인-API 인증 요청정보 암호화

본인확인-API인증 타입별 요청정보 (JSON)

휴대폰본인본인 (SMS / PASS)

SMS

이름	타입	설명	필수
serviceType	String	이용상품 코드 "telcoAuth" : 휴대폰본인확인 SMS/PASS 인증 서비스	O
providerId	String	서비스 제공 이동통신사 및 서비스 - SKT / SKTMVNO - KT / KTMVNO - LGU / LGUMVNO	O
reqAuthType	String	본인확인 인증Type 코드 (인증번호 전송 타입 또는 인증방법) - SMS	O
usageCode	String	서비스 이용 코드 - 01001 : 회원가입 - 01002 : 정보변경 - 01003 : ID찾기 - 01004 : 비밀번호찾기 - 01005 : 본인확인용 - 01007 : 상품구매/결제 - 01999 : 기타	O
username	String	사용자명	O
userPhone	String	사용자 핸드폰 번호	O
userBirthday	String	사용자 생년월일 (8자리) YYYYMMDD	O
userRegistSingleNumber	String	사용자 주민번호 뒤 자리 첫번째 숫자 해당 파라미터가 사용되는 경우 userGender와 userNation값은 무시됩니다.	

이름	타입	설명	필수
userGender	String	사용자 성별 - 1 : 남자 - 2 : 여자	O
userNation	String	사용자 내외국인 정보 - 0 : 내국인 - 1 : 외국인	O
sendMsg	String	SMS 전송 시 사용할 80Byte 이내 메시지. - NULL 또는 속성이 없을 경우 기본메시지 전송	
replyNumber	String	SMS 전송 시 사용할 송신번호 입력- NULL 또는 속성이 없을 경우 서비스에 등록된 이용기관 전화번호 또는 휴대폰본인확인 콜센터 번호 입력	
retTransferType	String	요청 대상의 결과 전송 타입 - 개인정보 서버 수신 모드 : MOKResult	O

JSON : MOKAuthInfo(인증요청 정보)

```
{
  "serviceType": "telcoAuth",
  "providerId": "${providerId}",
  "reqAuthType": "SMS",
  "usageCode": "${usageCode}",
  "userName": "${userName}",
  "userPhone": "${userPhone}",
  "userBirthday": "${userBirthday}",
  "userGender": "${userGender}",
  "userNation": "${userNation}",
  "retTransferType": "MOKResult"
}
```

위 JSON 데이터는 실제 사용 시 문자열(JSON string) 형태로 변환하여 사용해야 합니다.

PASS

이름	타입	설명	필수
serviceType	String	이용상품 코드 "telcoAuth" : 휴대폰본인확인 SMS/PASS 인증 서비스	O
providerId	String	서비스 제공 이동통신사 및 서비스 - SKT / SKTMVNO - KT / KTMVNO - LGU / LGUMVNO	O
reqAuthType	String	본인확인 인증Type 코드, 인증번호 전송 타입 또는 인증방법 - PASS	O
usageCode	String	서비스 이용 코드 - 01001 : 회원가입 - 01002 : 정보변경 - 01003 : ID찾기 - 01004 : 비밀번호찾기 - 01005 : 본인확인용 - 01007 : 상품구매/결제 - 01999 : 기타	O
Username	String	사용자명	O
userPhone	String	사용자 핸드폰 번호	O
retTransferType	String	요청 대상의 결과 전송 타입 - 개인정보 서버 수신 모드 : MOKResult	O

JSON : MOKAuthInfo(인증요청 정보)

```
{
  "serviceType": "telcoAuth",
  "providerId": "${providerId}",
  "reqAuthType": "PASS",
  "usageCode": "${usageCode}",
  "userName": "${userName}",
  "userPhone": "${userPhone}",
  "retTransferType": "MOKResult"
}
```

위 JSON 데이터는 실제 사용 시 문자열(JSON string) 형태로 변환하여 사용해야 합니다.

휴대폰본인본인 + 성인인증

SMS

이름	타입	설명	필수
serviceType	String	이용상품 코드 "telcoAuth-Adult" : 휴대폰본인확인 SMS 성인 인증 서비스	O
providerId	String	서비스 제공 이동통신사 및 서비스 - SKT / SKTMVNO - KT / KTMVNO - LGU / LGUMVNO	O
reqAuthType	String	본인확인 인증Type 코드 (인증번호 전송 타입 또는 인증방법) - SMS	O
usageCode	String	서비스 이용 코드 - 01016 : 민법 - 01026 : 청소년 보호법	O
username	String	사용자명	O
userPhone	String	사용자 핸드폰 번호	O
userBirthday	String	사용자 생년월일 (8자리) YYYYMMDD	O
userRegistSingleNumber	String	사용자 주민번호 뒤 자리 첫번째 숫자 해당 파라미터가 사용되는 경우 userGender와 userNation값은 무시됩니다.	
userGender	String	사용자 성별 - 1 : 남자 - 2 : 여자	O
userNation	String	사용자 내외국인 정보 - 0 : 내국인 - 1 : 외국인	O
sendMsg	String	SMS 전송 시 사용할 80Byte 이내 메시지. - NULL 또는 속성이 없을 경우 기본메시지 전송	
replyNumber	String	SMS 전송 시 사용할 송신번호 입력- NULL 또는 속성이 없을 경우 서비스에 등록된 이용기관 전화번호 또는 휴대폰본인확인 콜센터 번호 입력	

이름	타입	설명	필수
retTransferType	String	요청 대상의 결과 전송 타입 - 개인정보 서버 수신 모드 : MOKResult	O

JSON : MOKAuthInfo(인증요청 정보)

```
{
  "serviceType": "telcoAuth-Adult",
  "providerId": "${providerId}",
  "reqAuthType": "SMS",
  "usageCode": "${usageCode}",
  "userName": "${userName}",
  "userPhone": "${userPhone}",
  "userBirthday": "${userBirthday}",
  "userGender": "${userGender}",
  "userNation": "${userNation}",
  "retTransferType": "MOKResult"
}
```

위 JSON 데이터는 실제 사용 시 문자열(JSON string) 형태로 변환하여 사용해야 합니다.

휴대폰본인본인 + LMS

이름	타입	설명	필수
serviceType	String	이용상품 코드 - telcoAuth-LMS : 휴대폰본인확인 LMS 문자 안내 전송 서비스	O
providerId	String	서비스 제공 이동통신사 및 서비스 - SKT / SKTMVNO - KT / KTMVNO - LGU / LGUMVNO	O
reqAuthType	String	본인확인 인증Type 코드, 인증번호 전송 타입 또는 인증방법 - LMS	O

이름	타입	설명	필수
usageCode	String	서비스 이용 코드 - 01001 : 회원가입 - 01002 : 정보변경 - 01003 : ID찾기 - 01004 : 비밀번호찾기 - 01005 : 본인확인용 - 01007 : 상품구매/결제 - 01999 : 기타	O
username	String	사용자명	O
userPhone	String	사용자 핸드폰 번호	O
userBirthday	String	사용자 생년월일 (8자리) YYYYMMDD	O
userRegistSingleNumber	String	사용자 주민번호 뒤 자리 첫번째 숫자 해당 파라미터가 사용되는 경우 userGender와 userNation값은 무시됩니다.	
userGender	String	사용자 성별 - 1 : 남자 - 2 : 여자	O
userNation	String	사용자 내외국인 정보 - 0 : 내국인 - 1 : 외국인	O
sendMsg	String	LMS 전송 시 사용할 2000Byte 이내 메시지. NULL 또는 속성이 없을 경우 기본메시지 전송	
replyNumber	String	LMS 전송 시 사용할 송신번호 입력, NULL 또는 속성이 없을 경우 서비스에 등록된 이용기관 전화번호 또는 휴대폰본인확인 콜센터 번호 입력	
retTransferType	String	요청 대상의 결과 전송 타입- 개인정보 서버 수신 모드 : MOKResult	O

JSON : MOKAuthInfo(인증요청 정보)

```
{
  "serviceType": "telcoAuth-LMS",
  "providerId": ${providerId},
  "reqAuthType": "LMS",
  "usageCode": ${usageCode},
  "userName": ${userName},
```

```

"userPhone": ${userPhone},
"userBirthday": ${userBirthday},
"userGender": ${userGender},
"userNation": ${userNation},
"retTransferType": "MOKResult"
}

```

위 JSON 데이터는 실제 사용 시 문자열(JSON string) 형태로 변환하여 사용해야 합니다.

휴대폰본인확인 + ARS (ARS 점유인증)

이름	타입	설명	필수
serviceType	String	이용상품 코드 - telcoAuth-ARSAuth : 휴대폰본인확인 + ARS인증서비스	O
providerId	String	서비스 제공 이동통신사 및 서비스 - SKT / SKTMVNO - KT / KTMVNO - LGU / LGUMVNO	O
reqAuthType	String	본인확인 인증Type 코드, 인증번호 전송 타입 또는 인증방법 - ARS	O
arsCode	String	ARS인증을 사용하는 이용기관별로 등록된 ARS 코드 입력	O
usageCode	String	서비스 이용 코드 - 01001 : 회원가입 - 01002 : 정보변경 - 01003 : ID찾기 - 01004 : 비밀번호찾기 - 01005 : 본인확인용 - 01007 : 상품구매/결제 - 01999 : 기타	O
username	String	사용자명	O
userPhone	String	사용자 핸드폰 번호	O
userBirthday	String	사용자 생년월일 (8자리) YYYYMMDD	O

이름	타입	설명	필수
userGender	String	사용자 성별 - 1 : 남자 - 2 : 여자	O
userNation	String	사용자 내외국인 정보 - 0 : 내국인 - 1 : 외국인	O
replyNumber	String	SMS 전송 시 사용할 송신번호 입력 - NULL 또는 속성이 없을 경우 서비스에 등록된 이용기관 전화번호 또는 휴대폰본인확인 콜센터 번호 입력	
retTransferType	String	요청 대상의 결과 전송 타입 - 개인정보 서버 수신 모드 : MOKResult	O
extension	JSONObject	ARS 점유인증 요청시 이용기관별 추가정보 전달처리에 이용 추가정보 입력 extension/arsRequestMsg json "extension": { "arsRequestMsg": { "bankCode": "023" } }	

JSON : MOKAuthInfo(인증요청 정보)

```
{
  "serviceType": "telcoAuth-ARSAuth",
  "providerId": "${providerId}",
  "reqAuthType": "ARS",
  "arsCode" : "${arsCode}",
  "usageCode": "${usageCode}",
  "userName": "${userName}",
  "userPhone": "${userPhone}",
  "userBirthday": "${userBirthday}",
  "userGender": "${userGender}",
  "userNation": "${userNation}",
  "retTransferType": "MOKResult",
  "extension": {
    "arsRequestMsg": {
      "bankCode": "023"
    }
  }
}
```

```

}
}
}

```

위 JSON 데이터는 실제 사용 시 문자열(JSON string) 형태로 변환하여 사용해야 합니다.

SMS 점유인증

이름	타입	설명	필수
serviceType	String	이용상품 코드 - SMSAuth : SMS 점유인증 문자 안내 전송 서비스	O
providerId	String	서비스 제공 이동통신사 및 서비스 - SKT / SKTMVNO - KT / KTMVNO - LGU / LGUMVNO	O
reqAuthType	String	본인확인 인증Type 코드, 인증번호 전송 타입 또는 인증방법 - SMS	O
usageCode	String	서비스 이용 코드 - 01001 : 회원가입 - 01002 : 정보변경 - 01003 : ID찾기 - 01004 : 비밀번호찾기 - 01005 : 본인확인용 - 01007 : 상품구매/결제 - 01999 : 기타	O
username	String	사용자명	O
userPhone	String	사용자 핸드폰 번호	O
sendMsg	String	SMS 전송 시 사용할 80Byte 이내 메시지. - NULL 또는 속성이 없을 경우 기본메시지 전송	
replyNumber	String	SMS 전송 시 사용할 송신번호 입력 - NULL 또는 속성이 없을 경우 서비스에 등록된 이용기관 전화번호 또는 휴대폰본인확인 콜센터 번호 입력	
retTransferType	String	요청 대상의 결과 전송 타입 - 개인정보 서버 수신 모드 : MOKResult	O

JSON : MOKAuthInfo(인증요청 정보)

```
{
  "serviceType": "SMSAuth",
  "providerId": "${providerId}",
  "reqAuthType": "SMS",
  "usageCode": "${usageCode}",
  "userName": "${userName}",
  "userPhone": "${userPhone}",
  "retTransferType": "MOKResult"
}
```

위 JSON 데이터는 실제 사용 시 문자열(JSON string) 형태로 변환하여 사용해야 합니다.

암호화

본인확인-API 인증요청 정보(encryptMOKAuthInfo)

3 인증 요청정보 암호화

Step 1. AES 암호화 키와 IV 생성과 인증 요청 정보 암호화

- **Step 1-1.** 인증 요청 정보를 암호화하기 위해 AES 암호화 키(key)와 초기화 벡터(iv)를 생성합니다.

AES 암호화 키와 IV 생성

- 키와 IV는 난수 생성기를 사용해 무작위로 생성된 바이트 배열
- key는 32바이트 (AES 256bit 암호화용)
- iv는 16바이트 (AES CBC 모드에서 사용하는 IV)

예시) `byte[] aesKey = new byte[32]` `byte[] iv = new byte[16];`

⚠ 하드코딩된 값은 보안상 절대 사용하면 안됩니다, 매번 요청마다 새로 생성해야 합니다.

- **Step 1-2.** 인증요청 정보를 AES 암호화 (AES/CBC/PKCS5Padding)합니다.
 1. AES 키 및 IV 준비
 - 생성한 AES 키(`aesKey`)와 IV(`iv`)를 사용해서 암호화에 필요한 객체 준비
 2. AES 암호화 알고리즘 설정
 - AES/CBC/PKCS5Padding 방식으로 설정
 3. 인증 요청 본문 암호화
 - 인증요청 본문 (예: `MOKAuthInfo`)을 UTF-8로 인코딩 하여 바이트 배열(`byte[]`) 형태로 변환하여 암호화
 4. Base64 인코딩 처리
 - 암호화된 데이터를 그대로 전송하기 어려우므로, Base64 인코딩하여 저장 (예: `encData`)
- **Step 1-3.** 해시값을 생성 (SHA-256) 합니다.

📄 해시값 생성 (SHA-256)

- 암호화되기 전 인증 요청 본문 (예: `MOKAuthInfo`) 문자열을 SHA-256 해시로 처리
- 해시 결과는 Base64로 인코딩하여 문자열(hash)로 저장
- 이 해시값은 key + iv를 묶은 평문을 만들 때 사용

Step 2. AES 키(key)와 IV(iv)를 안전하게 전달하기 위한 추가 암호화

- **Step 2-1.** 공개키를 준비합니다.

📄 RSA 공개키

- 공개키는 X.509 포맷이며, Base64 디코딩 후 바이트 배열로 변환
- 공개키는 본인확인 거래요청정보(토큰) 요청 시 본인확인 서버로부터 응답받은 JSON의 `publicKey` 필드를 말한다.

```
{
  "encryptMOKToken" : "[RBj8YiVJ5e8l0ChfhE76+AJj ... pikDZMbeegv9r79uVUCmdA==",
  "publicKey" : "MIIBIjANBgkqhkiG ... N+Ey0zPfsWIDAQAB",
  "resultCode" : "2000",
```

```
"resultMsg" : "success"
}
```

• **Step 2-2.** 암호화 방식을 설정합니다.

1. RSA 암호화 방식 설정

- RSA with OAEP Padding 을 사용

 RSA 암호화 알고리즘 패딩 구성

- 해시 함수: SHA-256
- 마스킹 함수(MGF): MGF1 (SHA-256 기반)
- PSource: default 값 사용 (PSource.PSpecified.DEFAULT에 해당하는 값)
(SHA-256, MGF1, MGF1ParameterSpec("SHA-256"), PSource.PSpecified.DEFAULT)

 이 설정은 각 언어의 RSA 암호화 함수에서 패딩 옵션으로 명시되어야 하며, OAEP 모드를 지원하지 않는 RSA 함수는 사용할 수 없습니다.

• **Step 2-3.** key + iv를 결합하여 keyIv 바이트 배열(byte[])을 생성합니다.

1. key + iv 결합 (48바이트)

- AES 암호화에 사용된 key(32byte) 와 iv(16byte)를 붙여서 총 48바이트의 바이트 배열을 만듦

2. 첫 번째 구간 (0 ~ 31 바이트)

- 앞 32바이트에는 key(AES 암호화 키)값을 복사

3. 두 번째 구간 (32 ~ 47 바이트)

- 뒤 16 바이트에는 iv(AES 암호화 IV)값을 복사

4. Base64 인코딩 처리

• **Step 2-4.** 평문을 생성 (keyIvBase64 + '|' + hash) 합니다.

1. 최종 평문(encKey) 생성

- keyIv 바이트 배열을 Base64로 인코딩해서 문자열로 변환
- 그 문자열과 keyIv 의 SHA-256 해시 값을 | 기호로 이어 붙여 하나의 문자열로 만듦

• **Step 2-5.** 평문 데이터를 RSA 방식으로 암호화합니다.

1. 평문 문자열 변환

- Step 2-4에서 생성한 평문 문자열(encKey)을 UTF-8로 인코딩해서 바이트 배열로 변환

2. RSA 암호화 방식 설정 및 암호화 진행

- RSA/ECB/OAEPPadding 모드로 변환된 바이트 배열(byte[]) 암호화 진행
- **Step 2-6.** 암호화된 데이터를 조합하여 최종 전송 데이터를 생성합니다.
 1. RSA 암호화 결과 인코딩
 - Step 2-5에서 생성된 RSA 암호화 결과를 Base64로 인코딩해서 문자열로 변환
 2. 최종 데이터 조합 (encryptMOKAuthInfo)
 - 인코딩된 RSA 결과와 AES로 암호화한 인증 요청 데이터(encData)를 '|' 기호로 이어 붙여 최종 문자열 생성
 - 형식: 암호화된 encKey | encData

샘플은 JAVA로 제공됩니다.

```
public class MOKAuthInfoEncryption {
    // 인증요청 정보 암호화 메서드
    public static String encryptMOKAuthInfo(String authInfo, String publicKeyBase64) throws Exception {
        // Step 1: AES 암호화 키와 IV 생성
        SecureRandom secureRandom = new SecureRandom();
        byte[] aesKey = new byte[32]; // 32바이트 AES 키
        byte[] iv = new byte[16];    // 16바이트 IV
        secureRandom.nextBytes(aesKey);
        secureRandom.nextBytes(iv);

        // Step 1-2: AES 암호화 (AES/CBC/PKCS5Padding)
        Cipher aesCipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
        SecretKeySpec secretKeySpec = new SecretKeySpec(aesKey, "AES");
        IvParameterSpec ivSpec = new IvParameterSpec(iv);
        aesCipher.init(Cipher.ENCRYPT_MODE, secretKeySpec, ivSpec);

        byte[] encryptedData = aesCipher.doFinal(authInfo.getBytes(StandardCharsets.UTF_8));
        String encData = Base64.getEncoder().encodeToString(encryptedData);

        // Step 1-3: SHA-256 해시 생성
        MessageDigest sha256 = MessageDigest.getInstance("SHA-256");
        byte[] hashBytes = sha256.digest(authInfo.getBytes(StandardCharsets.UTF_8));
        String hash = Base64.getEncoder().encodeToString(hashBytes);

        // Step 2-3: key + iv 결합 (48바이트)
        byte[] keyIv = new byte[48];
        System.arraycopy(aesKey, 0, keyIv, 0, 32); // 앞 32바이트는 AES 키
        System.arraycopy(iv, 0, keyIv, 32, 16);    // 뒤 16바이트는 IV
    }
}
```

```

// Step 2-4: 평문 생성 (keyIvBase64 + '|' + hash)
String keyIvBase64 = Base64.getEncoder().encodeToString(keyIv);
String encKey = keyIvBase64 + "|" + hash;

// Step 2-1: RSA 공개키 준비
byte[] publicKeyBytes = Base64.getDecoder().decode(publicKeyBase64);
X509EncodedKeySpec keySpec = new X509EncodedKeySpec(publicKeyBytes);
KeyFactory keyFactory = KeyFactory.getInstance("RSA");
PublicKey publicKey = keyFactory.generatePublic(keySpec);

// Step 2-2 & 2-5: RSA 암호화 (RSA/ECB/OAEPPadding)
OAEPParameterSpec oaepParams = new OAEPParameterSpec(
    "SHA-256", "MGF1", MGF1ParameterSpec.SHA256, PSource.PSpecified.DEFAULT
);
Cipher rsaCipher = Cipher.getInstance("RSA/ECB/OAEPWithSHA-256AndMGF1Padding");
rsaCipher.init(Cipher.ENCRYPT_MODE, publicKey, oaepParams);

byte[] encryptedKeyIv = rsaCipher.doFinal(encKey.getBytes(StandardCharsets.UTF_8));
String encryptedKeyIvBase64 = Base64.getEncoder().encodeToString(encryptedKeyIv);

// Step 2-6: 최종 데이터 조합 (encryptMOKAuthInfo)
return encryptedKeyIvBase64 + "|" + encData;
}

public static void main(String[] args) throws Exception {
    Map<String, Object> data = new HashMap<>();
    data.put("serviceType", "serviceType");
    data.put("providerId", "providerId");
    data.put("reqAuthType", "reqAuthType");
    data.put("usageCode", "usageCode");
    data.put("userName", "userName");
    data.put("userPhone", "userPhone");
    data.put("userBirthday", "userBirthday");
    data.put("userGender", "userGender");
    data.put("userNation", "userNation");
    data.put("sendMsg", "sendMsg");
    data.put("replyNumber", "replyNumber");
    data.put("retTransferType", "retTransferType");
    ObjectMapper mapper = new ObjectMapper();
    // 인증 요청 데이터
    String target = mapper.writeValueAsString(data);
    // 서버에서 받은 공개키
    String publicKeyBase64 = "your_publicKey"

```

```
String encryptedResult = encryptAuthRequest(target, publicKeyBase64);  
System.out.println("Encrypted Result: " + encryptedResult);  
}  
}
```

5. 본인확인-API 인증(거래) 요청

본인확인 인증 요청

본인확인 토큰 발급 요청결과 encryptMOKToken 과 생성된 인증요청 정보 encryptMOKAuthInfo 를 이용하여 본인확인 인증 요청을 보냅니다.

메서드	URL	환경
POST	https://scert-dir.mobile-ok.com/agent/v1/auth/request	개발
POST	https://cert-dir.mobile-ok.com/agent/v1/auth/request	운영

1 요청

헤더

이름	설명	필수
Content-Type	Content-Type : application/json; charset=UTF-8 요청 데이터 타입	O

본문

이름	타입	설명	필수
encryptMOKToken	String	본인확인 토큰 발급 요청의 응답으로 받은 encryptMOKToken	O
encryptMOKAuthInfo	String	본인확인-API 인증요청 정보 - 서비스 별로 항목이 나누어져 있으니 필요한 서비스만 참고	O
siteUrl	String	본인확인 서비스 신청 URL	O

2 응답

본문

이름	타입	설명	필수
encryptMOKToken	String	암호화된 본인확인-API 거래 토큰	O
resultCode	String	응답 결과 코드 -2000일 경우 성공 -2000이 아닐 경우 에러 처리	O
resultMsg	String	응답 결과 코드가 2000이 아닐 경우 에러 메시지	O
resendCount	String	재시도 가능 횟수(요청 타입이 SMS인 경우)	

예제

요청

```
curl -X POST https://scert-dir.mobile-ok.com/agent/v1/auth/request \
-H "Content-Type: application/json; charset=UTF-8" \
-d '{
    "encryptMOKToken" : ${encryptMOKToken},
    "encryptMOKAuthInfo" : ${encryptMOKAuthInfo},
    "siteUrl" : ${siteUrl}
}'
```

응답 : 성공

```
{
  "encryptMOKToken" : "[RBj8YiVJ5e8l0ChfhE76+AJj ... pikDZMbeegv9r79uVUCmdA==",
  "resultCode" : "2000",
  "resultMsg" : "success",
```

```
"resendCount" : "2"
}
```

재요청

⚠ SMS 요청 타입의 본인확인 인증(거래) 요청에서 응답코드(결과코드)가 2000인 경우, 남은 재시도 횟수만큼 재시도가 가능합니다.

메서드	URL	인증 방식
POST	https://scert-dir.mobile-ok.com/agent/v1/auth/resend	

1 요청

헤더

이름	설명	필수
Content-Type	Content-Type : application/json; charset=UTF-8 요청 데이터 타입	O

본문

이름	타입	설명	필수
encryptMOKToken	String	본인확인 인증(거래) 요청의 응답으로 받은 encryptMOKToken	O
encryptMOKAuthInfo	String	본인확인-API 인증요청 정보 - 서비스 별로 항목이 나누어져 있으니 필요한 서비스만 참고	O
siteUrl	String	본인확인 서비스 신청 URL	O

2 응답

본문

이름	타입	설명	필수
encryptMOKToken	String	암호화된 본인확인-API 거래 토큰	O
resultCode	String	응답 결과 코드 -2000일 경우 성공 -2000이 아닐 경우 에러 처리	O
resultMsg	String	응답 결과 코드가 2000이 아닐 경우 에러 메시지	O
resendCount	String	재시도 가능 횟수(요청 타입이 SMS인 경우)	O

예제

요청

```
curl -X POST https://scert-dir.mobile-ok.com/agent/v1/auth/resend \
-H "Content-Type: application/json; charset=UTF-8" \
-d '{
    "encryptMOKToken" : ${encryptMOKToken},
    "encryptMOKAuthInfo" : ${encryptMOKAuthInfo},
    "siteUrl" : ${siteUrl}
}'
```

응답 : 성공

```
{
  "encryptMOKToken": "[P00/WNDlmNAntM+YVvSWZw ... KfVfd1xJlGr4P0Yhjg==",
  "resultCode": "2000",
  "resultMsg": "success",
```

```
"resendCount": "1"  
}
```


5-1. 본인확인-API ARS CALL 요청

본인확인 ARS CALL 요청

⚠ 휴대폰본인확인 + ARS (ARS 점유인증) 상품을 이용하는 이용기관만 참고

본인확인 인증(거래) 요청결과 encryptMOKToken 를 이용하여 본인확인 + ARS 인증요청을 보냅니다.

메서드	URL	환경
POST	https://scert-dir.mobile-ok.com/agent/v1/auth/call	개발
POST	https://cert-dir.mobile-ok.com/agent/v1/auth/call	운영

1 요청

헤더

이름	설명	필수
Content-Type	Content-Type : application/json; charset=UTF-8 요청 데이터 타입	O

본문

이름	타입	설명	필수
encryptMOKToken	String	본인확인 인증(거래) 요청의 응답으로 받은 encryptMOKToken	O

2 응답

본문

이름	타입	설명	필수
encryptMOKToken	String	암호화된 본인확인-API 거래 토큰	O
resultCode	String	응답 결과 코드 -2000일 경우 성공 -2000이 아닐 경우 에러 처리	O
resultMsg	String	응답 결과 코드가 2000이 아닐 경우 에러 메시지	O

예제

요청

```
curl -X POST https://scert-dir.mobile-ok.com/agent/v1/auth/call \
-H "Content-Type: application/json; charset=UTF-8" \
-d '{
    "encryptMOKToken" : ${encryptMOKToken}
}'
```

응답 : 성공

```
{
  "encryptMOKToken" : "[RBj8YiVJ5e8l0ChfhE76+AJj ... pikDZMbeegv9r79uVUCMdA==",
  "resultCode" : "2000",
  "resultMsg" : "success"
}
```

6. 본인확인-API 검증요청정보 암호화

4 본인확인 검증요청정보(토큰) 생성

⚠ 중요

키파일은 서버 내 안전한 장소에 보관

서비스 등록 후 발급된 `mok_keyInfo.dat` 를 안전한 서버 내 디렉토리에 보관

휴대폰본인확인-API 검증요청 정보

⚠ 인증번호가 있는 상품인 경우에만 암호화(`encryptMOKVerifyInfo`)

휴대폰본인확인 결과요청 원문 MOKVerifyInfo 생성

이름	타입	설명	필수
authNumber	String	휴대폰본인확인 서비스 인증번호 및 점유인증 인증정보 - SMS/LMS 인증번호 : SMS/LMS 문자로 전달받은 6자리 인증번호 - SMS 소유인증 : SMS 문자로 전달받은 인증번호	

암호화

본인확인 검증요청정보 암호화(`encryptMOKVerifyInfo`)

Step 1. AES 암호화 키와 IV 생성과 인증 요청 정보 암호화

- **Step 1-1.** 인증 요청 정보를 암호화하기 위해 AES 암호화 키(key)와 초기화 벡터(iv)를 생성합니다.

📄 AES 암호화 키와 IV 생성

- 키와 IV는 난수 생성기를 사용해 무작위로 생성된 바이트 배열
- key는 32바이트 (AES 256bit 암호화용)
- iv는 16바이트 (AES CBC 모드에서 사용하는 IV)

예시) `byte[] aesKey = new byte[32]` `byte[] iv = new byte[16];`

⚠ 하드코딩된 값은 보안상 절대 사용하면 안됩니다, 매번 요청마다 새로 생성해야 합니다.

- **Step 1-2.** 인증요청 정보를 AES 암호화 (AES/CBC/PKCS5Padding)합니다.
 1. AES 키 및 IV 준비
 - 생성한 AES 키(`aesKey`)와 IV(`iv`)를 사용해서 암호화에 필요한 객체 준비
 2. AES 암호화 알고리즘 설정
 - AES/CBC/PKCS5Padding 방식으로 설정
 3. 인증 요청 본문 암호화
 - 인증요청 본문 (예: `MOKVerifyInfo`) 을 UTF-8로 인코딩 하여 바이트 배열(`byte[]`) 형태로 변환하여 암호화
 4. Base64 인코딩 처리
 - 암호화된 데이터를 그대로 전송하기 어려우므로, Base64 인코딩하여 저장 (예: `encData`)
- **Step 1-3.** 해시값을 생성 (SHA-256) 합니다.

📄 해시값 생성 (SHA-256)

- 암호화되기 전 인증 요청 본문 (예: `MOKVerifyInfo`) 문자열을 SHA-256 해시로 처리
- 해시 결과는 Base64로 인코딩하여 문자열(hash)로 저장
- 이 해시값은 key + iv를 묶은 평문을 만들 때 사용

Step 2. AES 키(key)와 IV(iv)를 안전하게 전달하기 위한 추가 암호화

- **Step 2-1.** 공개키를 준비합니다.

📄 RSA 공개키

- 공개키는 X.509 포맷이며, Base64 디코딩 후 바이트 배열로 변환
- 공개키는 본인확인 거래요청정보(토큰) 요청 시 본인확인 서버로부터 응답받은 JSON의 `publicKey` 필드를 말한다.

```
{
  "encryptMOKToken" : "[RBj8YiVJ5e8l0ChfhE76+AJj ... pikDZMbeegv9r79uVUcMdA==",
  "publicKey" : "MIIBIjANBgkqhkiG ... N+Ey0zPfsWIDAQAB",
  "resultCode" : "2000",
  "resultMsg" : "success"
}
```

• **Step 2-2.** 암호화 방식을 설정합니다.

1. RSA 암호화 방식 설정

- RSA with OAEP Padding 을 사용

 RSA 암호화 알고리즘 패딩 구성

- 해시 함수: SHA-256
- 마스킹 함수(MGF): MGF1 (SHA-256 기반)
- PSource: default 값 사용 (PSource.PSpecified.DEFAULT에 해당하는 값)
(SHA-256, MGF1, MGF1ParameterSpec("SHA-256"), PSource.PSpecified.DEFAULT)

 이 설정은 각 언어의 RSA 암호화 함수에서 패딩 옵션으로 명시되어야 하며, OAEP 모드를 지원하지 않는 RSA 함수는 사용할 수 없습니다.

• **Step 2-3.** key + iv를 결합하여 keyIv 바이트 배열(byte[])을 생성합니다.

1. key + iv 결합 (48바이트)

- AES 암호화에 사용된 key(32byte) 와 iv(16byte)를 붙여서 총 48바이트의 바이트 배열을 만들

2. 첫 번째 구간 (0 ~ 31 바이트)

- 앞 32바이트에는 key(AES 암호화 키)값을 복사

3. 두 번째 구간 (32 ~ 47 바이트)

- 뒤 16 바이트에는 iv(AES 암호화 IV)값을 복사

4. Base64 인코딩 처리

• **Step 2-4.** 평문을 생성 (keyIvBase64 + '|' + hash) 합니다.

1. 최종 평문(encKey) 생성

- keyIv 바이트 배열을 Base64로 인코딩해서 문자열로 변환
- 그 문자열과 keyIv 의 SHA-256 해시 값을 | 기호로 이어 붙여 하나의 문자열로 만들

• **Step 2-5.** 평문 데이터를 RSA 방식으로 암호화합니다.

1. 평문 문자열 변환
 - Step 2-4에서 생성한 평문 문자열(encKey)을 UTF-8로 인코딩해서 바이트 배열로 변환
2. RSA 암호화 방식 설정 및 암호화 진행
 - RSA/ECB/OAEPPadding 모드로 변환된 바이트 배열(byte[]) 암호화 진행
- **Step 2-6.** 암호화된 데이터를 조합하여 최종 전송 데이터를 생성합니다.
 1. RSA 암호화 결과 인코딩
 - Step 2-5에서 생성된 RSA 암호화 결과를 Base64로 인코딩해서 문자열로 변환
 2. 최종 데이터 조합 (encryptMOKVerifyInfo)
 - 인코딩된 RSA 결과와 AES로 암호화한 인증 요청 데이터(encData)를 '|' 기호로 이어 붙여 최종 문자열 생성
 - 형식: 암호화된 encKey | encData

```
public class MOKVerifyInfoEncryption {

    // MOKVerifyInfo 클래스 (검증요청 정보)
    public static class MOKVerifyInfo {
        private String authNumber;

        public MOKVerifyInfo(String authNumber) {
            this.authNumber = authNumber;
        }

        // JSON 문자열로 변환
        public String toJson() {
            JSONObject json = new JSONObject();
            json.put("authNumber", authNumber);
            return json.toString();
        }
    }

    // 검증요청 정보 암호화 메서드
    public static String encryptMOKVerifyInfo(MOKVerifyInfo verifyInfo, String publicKeyBase64) throws Exception {
        // 인증번호가 없는 경우 암호화하지 않음 (예: 휴대폰본인확인 PASS인증, ARS 점유인증)
        if (verifyInfo == null || verifyInfo.authNumber == null || verifyInfo.authNumber.isEmpty()) {
            return "";
        }

        // Step 1: AES 암호화 키와 IV 생성
        SecureRandom secureRandom = new SecureRandom();
```

```

byte[] aesKey = new byte[32]; // 32바이트 AES 키
byte[] iv = new byte[16]; // 16바이트 IV
secureRandom.nextBytes(aesKey);
secureRandom.nextBytes(iv);

// Step 1-2: AES 암호화 (AES/CBC/PKCS5Padding)
Cipher aesCipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
SecretKeySpec secretKeySpec = new SecretKeySpec(aesKey, "AES");
IvParameterSpec ivSpec = new IvParameterSpec(iv);
aesCipher.init(Cipher.ENCRYPT_MODE, secretKeySpec, ivSpec);

String verifyInfoJson = verifyInfo.toJson();
byte[] encryptedData = aesCipher.doFinal(verifyInfoJson.getBytes(StandardCharsets.UTF_8));
String encData = Base64.getEncoder().encodeToString(encryptedData);

// Step 1-3: SHA-256 해시 생성
MessageDigest sha256 = MessageDigest.getInstance("SHA-256");
byte[] hashBytes = sha256.digest(verifyInfoJson.getBytes(StandardCharsets.UTF_8));
String hash = Base64.getEncoder().encodeToString(hashBytes);

// Step 2-3: key + iv 결합 (48바이트)
byte[] keyIv = new byte[48];
System.arraycopy(aesKey, 0, keyIv, 0, 32); // 앞 32바이트는 AES 키
System.arraycopy(iv, 0, keyIv, 32, 16); // 뒤 16바이트는 IV

// Step 2-4: 평문 생성 (keyIvBase64 + '|' + hash)
String keyIvBase64 = Base64.getEncoder().encodeToString(keyIv);
String encKey = keyIvBase64 + "|" + hash;

// Step 2-1: RSA 공개키 준비
byte[] publicKeyBytes = Base64.getDecoder().decode(publicKeyBase64);
X509EncodedKeySpec keySpec = new X509EncodedKeySpec(publicKeyBytes);
KeyFactory keyFactory = KeyFactory.getInstance("RSA");
PublicKey publicKey = keyFactory.generatePublic(keySpec);

// Step 2-2 & 2-5: RSA 암호화 (RSA/ECB/OAEPPadding)
OAEPParameterSpec oaepParams = new OAEPParameterSpec(
    "SHA-256", "MGF1", MGF1ParameterSpec.SHA256, PSource.PSpecified.DEFAULT
);
Cipher rsaCipher = Cipher.getInstance("RSA/ECB/OAEPWithSHA-256AndMGF1Padding");
rsaCipher.init(Cipher.ENCRYPT_MODE, publicKey, oaepParams);

```

```

byte[] encryptedKeyIv = rsaCipher.doFinal(encKey.getBytes(StandardCharsets.UTF_8));
String encryptedKeyIvBase64 = Base64.getEncoder().encodeToString(encryptedKeyIv);

// Step 2-6: 최종 데이터 조합 (encryptMOKVerifyInfo)
return encryptedKeyIvBase64 + "|" + encData;
}

// 테스트용 메인 메서드
public static void main(String[] args) throws Exception {
    // 예시: SMS/LMS/SMS 소유인증용 인증번호
    MOKVerifyInfo verifyInfo = new MOKVerifyInfo("123456"); // 6자리 인증번호
    // 공개키 (서버에서 받은 X.509 Base64 공개키)
    String publicKeyBase64 = "your_publicKey";

    // 암호화 실행
    String encryptedResult = encryptMOKVerifyInfo(verifyInfo, publicKeyBase64);
    System.out.println("Encrypted MOKVerifyInfo: " + encryptedResult);

    // 예시: 인증번호가 없는 경우 (예: 휴대폰본인확인 PASS인증, ARS 점유인증)
    MOKVerifyInfo emptyVerifyInfo = new MOKVerifyInfo("");
    String emptyResult = encryptMOKVerifyInfo(emptyVerifyInfo, publicKeyBase64);
    System.out.println("Encrypted MOKVerifyInfo (Empty): " + emptyResult);
}
}

```


7. 본인확인-API 인증결과 요청

본인확인 인증결과 요청 JSON 생성

인증 결과 요청

메서드	URL	환경
POST	https://scert-dir.mobile-ok.com/agent/v1/confirm/request	개발
POST	https://cert-dir.mobile-ok.com/agent/v1/confirm/request	운영

1 요청

헤더

이름	설명	필수
Content-Type	Content-Type : application/json; charset=UTF-8 요청 데이터 타입	O

본문

이름	타입	설명	필수
encryptMOKToken	String	본인확인-API 인증요청 또는 ARS전화요청 후 수신받은 암호화된 토큰	O
encryptMOKVerifyInfo	String	암호화된 휴대폰본인확인 MOKVerifyInfo 인증정보	

2 응답

본문

이름	타입	설명	필수
resultCode	String	응답 결과 코드 -2000일 경우 성공 -2000이 아닐 경우 에러 처리	O
resultMsg	String	응답 결과 코드가 2000이 아닐 경우 에러 메시지	O
encryptMOKResult	String	암호화된 본인확인 인증결과 데이터(본인확인-API 표준응답)	O
retryCount	String	검증결과 요청 재 시도 가능횟수, 1번 잘못된 인증결과 시도 후 2 (남은 재시도 횟수) 응답	

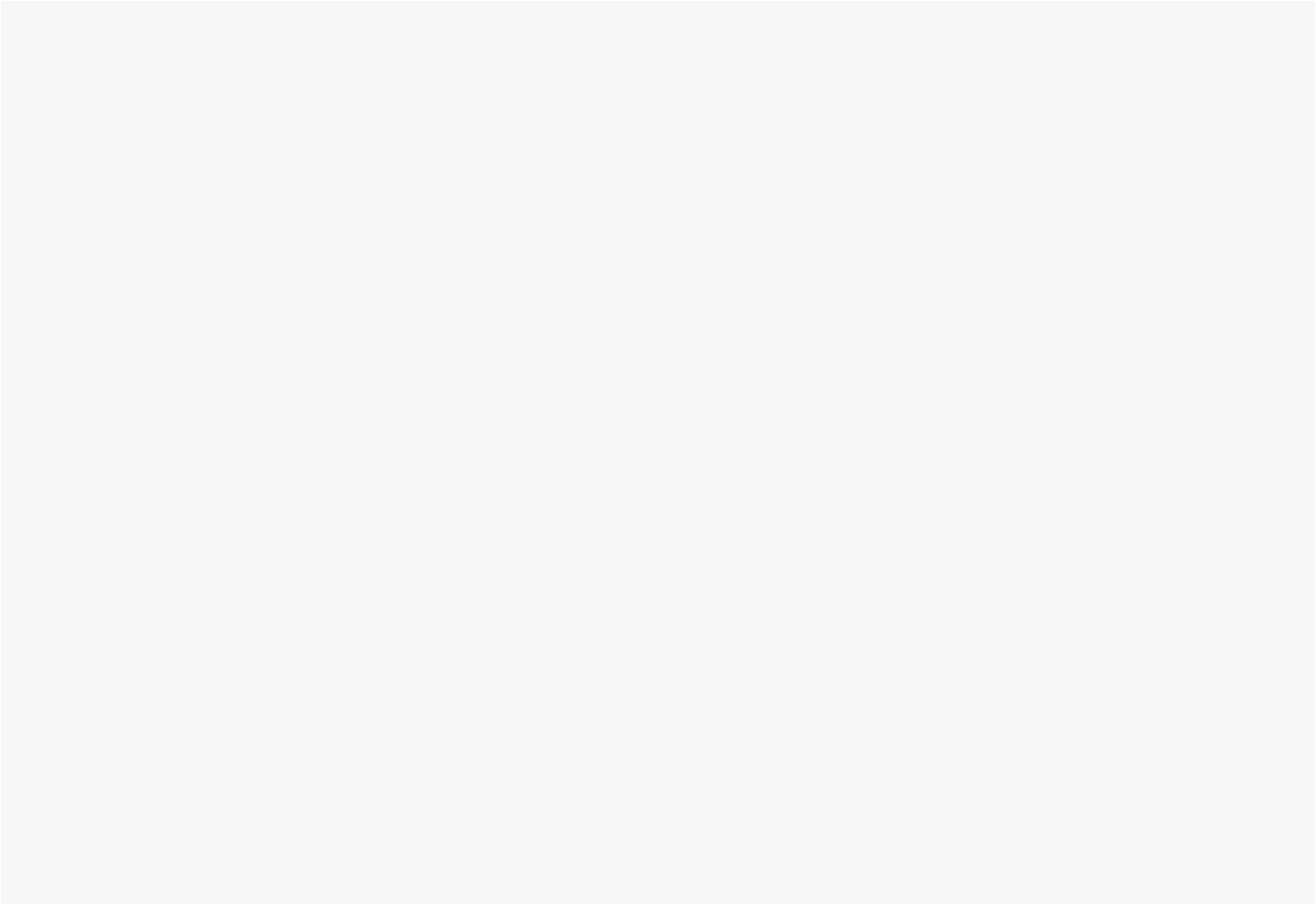
예제

요청

```
curl -X POST https://scert-dir.mobile-ok.com/agent/v1/confirm/request \
-H "Content-Type: application/json; charset=UTF-8" \
-d '{
    "encryptMOKToken" : ${encryptMOKToken},
    "encryptMOKVerifyInfo" : ${encryptMOKVerifyInfo},
  }'
```

응답 : 성공

```
{
  "encryptMOKResult" : "ln0XRGpb//+vzabdYpcjj0lmivt8qI ... 3eUZgz1jaqu0ZRXz8=",
  "resultCode" : "2000",
  "resultMsg" : "success"
}
```



8. 본인확인-API 개인정보 복호화

5 본인확인 개인정보 복호화



중요

키파일은 서버 내 안전한 장소에 보관

서비스 등록 후 발급된 `mok_keyInfo.dat` 를 안전한 서버 내 디렉토리에 보관

휴대폰본인확인 인증결과 encryptMOKResult 복호화

Step 1. 암호화된 결과 데이터 준비

- **Step 1-1.** 서버로부터 받은 암호화된 인증결과(encryptMOKResult)를 준비합니다.
- **Step 1-2.** encryptMOKResult를 '|'로 분리하여 각각의 데이터를 추출합니다.



Info

- encryptMOKResult는 두 부분으로 구성되며, '|' 구분자로 나누어짐
 - 첫 번째 부분(encryptKeyIvHashData) : RSA로 암호화된 키와 IV 정보 (Base64 인코딩)
 - 두 번째 부분(encryptResultData) : AES로 암호화된 실제 인증결과 데이터 (Base64 인코딩)

Step 2. RSA 복호화로 AES 키와 IV 추출

- **Step 2-1.** RSA 개인키를 준비합니다.
 1. 개인키를 Base64 디코딩 하여 바이트 배열(byte[]) 형태로 읽어서 준비
- **Step 2-2.** RSA 복호화 설정을 구성합니다.
 1. RSA 암호화 방식 설정



RSA 암호화 알고리즘 패딩 구성

- 해시 함수: SHA-256

- 마스크 함수(MGF): MGF1 (SHA-256 기반)
- PSource: default 값 사용 (PSource.PSpecified.DEFAULT에 해당하는 값)
(SHA-256, MGF1, MGF1ParameterSpec("SHA-256"), PSource.PSpecified.DEFAULT)

⚠ 이 설정은 각 언어의 RSA 암호화 함수에서 패딩 옵션으로 명시되어야 하며, OAEP 모드를 지원하지 않는 RSA 함수는 사용할 수 없습니다.

- **Step 2-3.** RSA 복호화를 수행합니다.
 1. encryptKeyIvHashData를 Base64 디코딩하여 바이트 배열로 변환
 2. RSA 복호화를 통해 평문(keyIvHashData)을 얻음
- **Step 2-4.** keyIvHashData를 '|'로 분리하여 base64KeyIv와 hashData를 추출합니다.

Info

- 개인키(ClientPrivateKey) : 키파일 內 이용기관의 암호/복호화용 키
- 개인키는 PKCS#8 포맷으로 관리됨
- 평문(keyIvHashData)은 '|'로 구분된 두 부분으로 구성
 - 첫 번째 부분: Base64로 인코딩된 keyIv (AES 키 + IV)
 - 두 번째 부분: 원문 데이터의 SHA-256 해시값 (Base64 인코딩)

Step 3. AES 키와 IV 준비

- **Step 3-1.** base64KeyIv를 Base64 디코딩하여 48바이트 배열(keyIv)로 변환합니다.
- **Step 3-2.** keyIv에서 AES 키와 IV를 추출합니다.

Info

- 첫 32바이트: AES-256 암호화 키
- 다음 16바이트: 초기화 벡터 (iv)

Step 4. AES 복호화

- **Step 4-1.** AES/CBC/PKCS5Padding 모드로 위에서 추출한 AES 키와 IV를 이용하여 데이터를 복호화합니다.

Step 5. 결과 확인 및 검증

- **Step 5-1.** 복호화된 결과를 JSON, UTF-8 문자열로 변환
 1. 복호화된 평문 데이터를 SHA-256으로 해싱.
 2. 해시 결과를 Base64로 인코딩하여 Step 2-3에서 얻은 hashData와 비교.



중요

- JSON 형태로 복호화가 되는지 검증 필수
- 무결성 검증 : 데이터 변조 여부 확인

```
{
  "siteId" : "이용기관 ID",
  "clientTxId" : "이용기관 거래 ID",
  "txId" : "본인확인 거래 ID",
  "providerId" : "서비스제공자(인증사업자) ID",
  "serviceType" : "이용 서비스 유형",
  "ci" : "사용자 CI",
  "di" : "사용자 DI",
  "userName" : "사용자 이름",
  "userPhone" : "사용자 전화번호",
  "userBirthday" : "사용자 생년월일",
  "userGender" : "사용자 성별 (1: 남자, 2: 여자)",
  "userNation" : "사용자 국적 (0: 내국인, 1: 외국인)",
  "reqAuthType" : "본인확인 인증 종류",
  "reqDate" : "본인확인 요청 시간",
  "issuer" : "본인확인 인증 서버",
  "issueDate" : "본인확인 인증 시간"
  ...
}
```

⚠️ 중요: 이용기관 서비스 기능 처리

- 개인정보 검증 및 CI 확인
 - 이용기관에서 수신한 개인정보의 검증 처리

- 이용기관에서 수신한 CI 확인 처리
- 개인정보 관리
 - 복호화된 개인정보는 DB등 서버저장 매체에 안전하게 저장하여야 하며, 본인확인 과정에서 획득한 정보로 저장
 - 개인정보를 웹 브라우저로 전송할 경우, 외부 해킹 및 유출 방지를 위해 철저한 보안 처리 필수

샘플은 JAVA로 제공됩니다.

```
public class MOKResultDecryption {

    public static String decryptMOKResult(String encryptMOKResult, byte[] privateKey) throws Exception {
        // Step 1: 암호화된 결과 데이터 준비
        // Step 1-1 & 1-2: encryptMOKResult를 '|'로 분리
        String[] parts = encryptMOKResult.split("\\|");
        if (parts.length != 2) {
            throw new IllegalArgumentException("Invalid encryptMOKResult format");
        }
        String encryptKeyIvHashData = parts[0];
        String encryptResultData = parts[1];

        // Step 2: RSA 복호화로 AES 키와 IV 추출
        // Step 2-1 & 2-2: RSA 개인키 및 설정 준비
        Cipher rsaCipher = Cipher.getInstance("RSA/ECB/OAEPWithSHA-256AndMGF1Padding");
        KeyFactory keyFactory = KeyFactory.getInstance("RSA");
        OAEPParameterSpec oaepParams = new OAEPParameterSpec("SHA-256", "MGF1", MGF1ParameterSpec.SHA256, PSource.PSpecified.DEFAULT);
        PKCS8EncodedKeySpec privateKeySpec = new PKCS8EncodedKeySpec(privateKey);
        PrivateKey rsaPrivateKey = keyFactory.generatePrivate(privateKeySpec);

        // Step 2-3: RSA 복호화
        rsaCipher.init(Cipher.DECRYPT_MODE, rsaPrivateKey, oaepParams);
        String keyIvHashData = new String(
            rsaCipher.doFinal(Base64.getDecoder().decode(encryptKeyIvHashData)),
            StandardCharsets.UTF_8
        );

        // Step 2-4: keyIvHashData 분리
        String[] keyIvHashParts = keyIvHashData.split("\\|");
        if (keyIvHashParts.length != 2) {
```

```

        throw new IllegalArgumentException("Invalid keyIvHashData format");
    }
    String base64KeyIv = keyIvHashParts[0];
    String hashData = keyIvHashParts[1];

    // Step 3: AES 키와 IV 준비
    // Step 3-1 & 3-2: keyIv 디코딩 및 키/IV 추출
    byte[] keyIv = Base64.getDecoder().decode(base64KeyIv);
    if (keyIv.length != 48) {
        throw new IllegalArgumentException("Invalid keyIv length");
    }
    byte[] key = new byte[32];
    byte[] iv = new byte[16];
    System.arraycopy(keyIv, 0, key, 0, 32); // AES 키
    System.arraycopy(keyIv, 32, iv, 0, 16); // IV

    // Step 4: AES 복호화
    // Step 4-1: AES 설정
    Cipher aesCipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
    SecretKeySpec secretKeySpec = new SecretKeySpec(key, "AES");
    IvParameterSpec ivSpec = new IvParameterSpec(iv);
    aesCipher.init(Cipher.DECRYPT_MODE, secretKeySpec, ivSpec);

    // Step 4-2: AES 복호화
    byte[] decryptedData = aesCipher.doFinal(Base64.getDecoder().decode(encryptResultData));
    String result = new String(decryptedData, StandardCharsets.UTF_8);

    // Step 5: 결과 검증
    // Step 5-1: 무결성 검증 (SHA-256 해시 비교)
    MessageDigest digest = MessageDigest.getInstance("SHA-256");
    byte[] resultHashBytes = digest.digest(result.getBytes(StandardCharsets.UTF_8));
    String computedHash = Base64.getEncoder().encodeToString(resultHashBytes);
    if (!computedHash.equals(hashData)) {
        throw new SecurityException("Hash verification failed: Data integrity compromised");
    }

    return result;
}

// 테스트용 메인 메서드
public static void main(String[] args) throws Exception {
    // 예시 입력값

```



```
String encryptMOKResult = "base64EncryptedKeyIvHash|base64EncryptedResultData"; // 실제 값으로 대체  
byte[] privateKey = Base64.getDecoder().decode("your_base64_pkcs8_private_key"); // 실제 개인키로 대체
```

```
try {  
    String decryptedResult = decryptMOKResult(encryptMOKResult, privateKey);  
    System.out.println("Decrypted JSON Result: " + decryptedResult);  
} catch (Exception e) {  
    System.err.println("Decryption failed: " + e.getMessage());  
}
```

```
}
```

```
}
```